

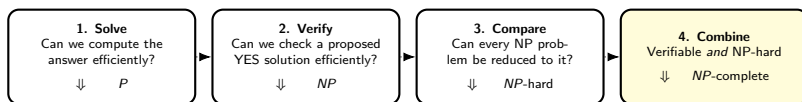
NP-Completeness and Reduction Techniques

How Efficient Transformations Prove Computational Hardness

Most. Sonia Khatun

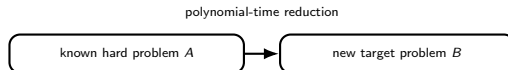
BSc in Department of Computer Science and Engineering,
Bangladesh University of Engineering and Technology (BUET)

Why Do We Need NP-Completeness?



The central proof question

How can we transfer the known hardness of one problem to a new problem?

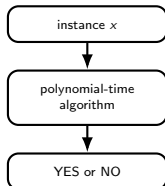


The dotted arrows show the learning sequence, not set containment.

Start with P and NP

P: solve efficiently

Decision problems solvable by a deterministic algorithm in polynomial time.

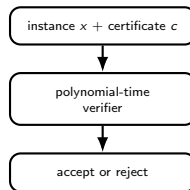


Example: reachability can be solved by BFS or DFS.

NP: verify efficiently

There are a polynomial-time verifier V and polynomial p such that, for every x ,

$$x \in L \iff \exists c : \begin{matrix} |c| \leq p(|x|), \\ V(x, c) = 1. \end{matrix}$$



For 3-COLOR, c assigns a color to every vertex; scan all edges to verify it.

$P \subseteq NP$: a polynomial-time solver can serve as a verifier and ignore the certificate.

Important: NP means "nondeterministic polynomial time," not "non-polynomial." Whether $P = NP$ is open.

From NP-Hard to NP-Complete

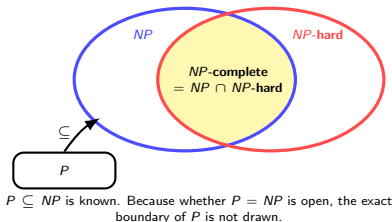
NP-hard: hard for all of NP

A problem H is NP-hard if

$$\forall L \in NP, \quad L \leq_p H.$$

It need not itself belong to NP.

NP-complete: both properties

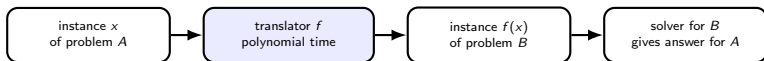


$$H \in NP \quad \text{and} \quad H \text{ is NP-hard.}$$

These are the hardest efficiently verifiable decision problems.

If one NP-complete problem is in P , then $P = NP$.

Polynomial-Time Reduction: an Answer-Preserving Translator



Formal meaning

$A \leq_p B$ means there exists a polynomial-time computable function f such that

$$\forall x, \quad x \in A \iff f(x) \in B.$$

Therefore, a solver for B would also solve A : B is at least as hard as A .

To prove B hard: known hard
 $A \longrightarrow$ target B .

Tiny example: $\text{EVEN} \leq_p \text{MULTIPLE-OF-4}$

Use $f(n) = 2n$. Then

$$n \text{ is even} \iff 4 \mid 2n.$$

n	$f(n)$	preserved?
6: YES	12: YES	YES
7: NO	14: NO	YES

This illustrates the mechanics; a hardness proof starts from a known hard problem.

Reusable Recipe: Proving a Problem NP-Complete

1. Membership in NP

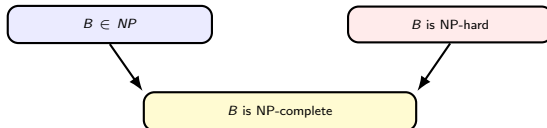
- ▶ State a polynomial-size certificate.
- ▶ Give a polynomial-time verifier.

$B \in NP$

2. NP-hardness

1. Choose a known NP-complete problem A .
2. Construct f in polynomial time.
3. Prove $x \in A \iff f(x) \in B$.

$A \leq_p B \implies B \text{ is NP-hard}$



Direction mnemonic: the known hard problem goes *into* the target problem.

Worked Reduction: 3-SAT $\leq_p$ 3-COLOR

Source: 3-SAT

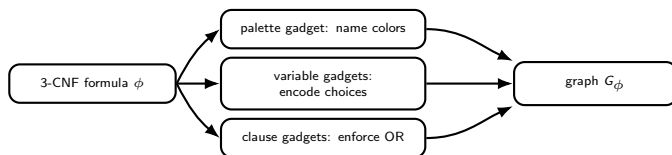
Given a 3-CNF formula

$$\phi = C_1 \wedge \cdots \wedge C_m,$$

is there a truth assignment satisfying every clause?

Target: 3-COLOR

Given a graph G , can its vertices be colored with three colors so that adjacent vertices have different colors?



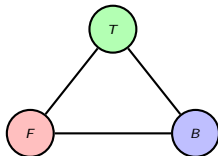
Design invariant: ϕ is satisfiable $\iff$ the constructed graph G_ϕ is 3-colorable.

This is one illustration of the general proof recipe—not the definition of a reduction.

Encoding Truth with Colors

Palette gadget

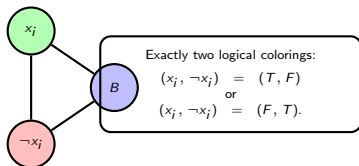
The triangle forces three distinct colors. We name them T , F , and B (base/helper).



three vertices $\Rightarrow$ three meanings

Variable gadget

Connect x_i and $\neg x_i$ to each other and to B .



Exactly one of x_i and $\neg x_i$ receives color T .

Constructing the Exact Clause Gadget

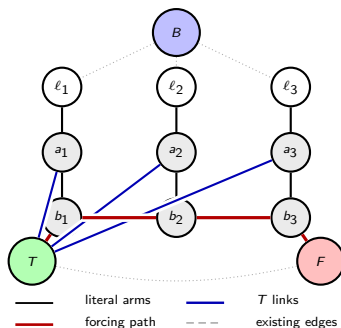
For $C = (\ell_1 \vee \ell_2 \vee \ell_3)$, add six fresh vertices a_i, b_i and 13 edges:

$$(\ell_i, a_i), (a_i, b_i), (T, a_i) \\ \text{for } i = 1, 2, 3,$$

$$(T, b_1), (b_1, b_2), \\ (b_2, b_3), (b_3, F).$$

Exact property

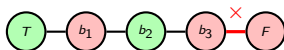
It extends to a 3-coloring exactly when at least one input literal has color T .



Why the Clause Gadget Encodes OR

All inputs are F : impossible

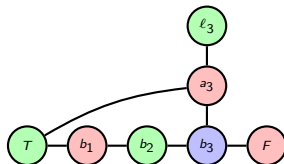
If every $\ell_i = F$, each a_i is forced to B .
Hence each b_i can use only T or F .



The path must alternate T, F, T, F, F ; its last edge has equal colors.

At least one input is T : possible

For a true input ℓ_i , set $a_i = F$ and $b_i = B$;
complete the remaining vertices legally.



Shown for $\ell_3 = T$; analogous completions handle either other true input.

F, F, F : no extension; all other seven input patterns: an extension exists.

Evaluating One Complete Formula

Use

$$\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3).$$

Clause input ports

$$C_1 : (x_1, \neg x_2, x_3),$$

$$C_2 : (\neg x_1, x_2, \neg x_3).$$

Each clause receives its own six fresh internal vertices.

One satisfying assignment

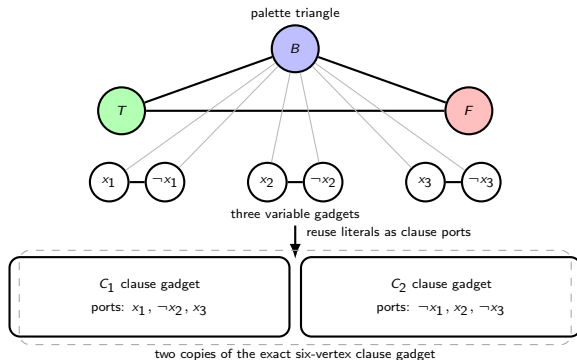
$$x_1 = T, \quad x_2 = T, \quad x_3 = F.$$

Thus

$$C_1 = (T, F, F), \quad C_2 = (F, T, T).$$

Both clause gadgets can be extended.

Assembling the Formula Graph



The palette and literal vertices are shared; each clause has fresh internal vertices.

Correctness and Complexity of the Reduction

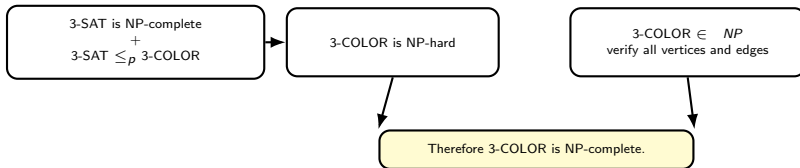
ϕ satisfiable $\Rightarrow G_\phi$ 3-colorable

- ▶ Give each literal its truth color.
- ▶ Every clause has a true literal.
- ▶ Therefore every clause gadget extends to a legal coloring.

G_ϕ 3-colorable $\Rightarrow \phi$ satisfiable

- ▶ Read $x_i = \text{true}$ exactly when vertex x_i has color T .
- ▶ Every colorable clause gadget has a T input.
- ▶ Therefore every clause is satisfied.

ϕ satisfiable $\iff G_\phi$ 3-colorable

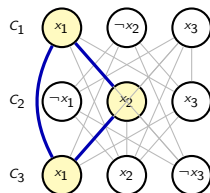


Construction size: $|V| = 3 + 2n + 6m$ and $|E| = 3 + 3n + 13m$, hence polynomial.

A Second Reduction Pattern: 3-SAT $\leq_p$ CLIQUE

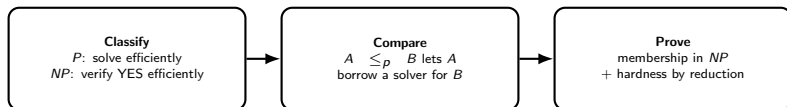
$$\phi' = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \neg x_3).$$

- Create one vertex for each literal occurrence.
- Put vertices in clause groups.
- Connect vertices from different clauses when they are not contradictory.
- Ask for a clique of size m .



One compatible literal from each clause forms a clique.

Takeaway



One reusable sentence

To prove a target B NP-complete, show $B \in NP$ and reduce a known NP-complete problem A to B in polynomial time while preserving YES/NO answers.

References

Primary references

- ▶ Michael Sipser, *Introduction to the Theory of Computation*.
- ▶ Cormen, Leiserson, Rivest, Stein, *Introduction to Algorithms*.
- ▶ CMU 15-451/651 lecture notes on NP-completeness.
- ▶ Princeton COS 423 lecture notes, *Polynomial-Time Reductions*.

The exact six-vertex clause gadget and its 13-edge construction follow the Princeton reduction notes.

Thank You

Questions?